CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G1
System Integration and Architecture

System Design Document (SDD)

Project Title: RentEasy

Prepared By: Nuevas, Josh Anton K.

Version: 0.3

Date: 05/20/2026

Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	02/21/2026	Josh Anton K. Nuevas	Initial draft	Draft
0.2	02/28/2026	Josh Anton K. Nuevas	Added contents for System Architecture, API Contract and Communication Flow, Database Design, UI/UX Design, and Project Plan.	Draft
0.3	05/20/2026	Josh Anton K. Nuevas	Finalized database design, API contract, requirements, technology stack, ERD, component diagram, project timeline, UI/UX documentation, Google authentication, and owner listing flow based on the completed RentEasy implementation.	Final

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY

1.1 Project Overview

RentEasy is a rental marketplace system designed to let users browse, list, and rent items through web and mobile interfaces. The system focuses on rental-based transactions rather than traditional product selling. Users can view approved rental listings, add items to a cart, set rental days, and proceed to checkout through PayMongo. Product owners can submit rental listings for approval, while administrators can review, approve, reject, and remove listings.

1.2 Objectives

System Integration:

Build a connected rental platform using a Spring Boot backend, React web frontend, Kotlin Android mobile application, Supabase PostgreSQL database, and PayMongo checkout integration.

Backend API Communication:

Provide REST API endpoints for email/password authentication, Google authentication, product listing management, owner listing retrieval, cart management, listing approval, profile management, order management, and PayMongo checkout session creation.

User Experience:

Deliver a responsive web interface and Android mobile interface for browsing products, managing cart items, submitting listings, and accessing admin functions.

Database Organization:

Store regular users, admin accounts, products, cart items, rental orders, and rental order items in a structured Supabase PostgreSQL database with clear table relationships.

Admin Control:

Allow administrators to manage listings by approving, rejecting, viewing, and deleting rental products.

1.3 Scope

Included Features:

- User registration and login using email/password, with Google sign-in support for regular user accounts
- JWT-based authentication for protected backend endpoints
- Separate admin accounts stored outside the regular users table
- Approved rental catalog display
- Product detail viewing
- Frontend product search/filtering
- Product listing submission by users
- Pending listing approval flow
- Admin dashboard
- Admin product viewing and removal
- Shopping cart management: add, remove, view, and update rental days
- Checkout form with delivery/contact information
- PayMongo checkout session creation
- Order confirmation display after checkout return
- User profile display and frontend profile updates
- Profile picture change on the web frontend
- Responsive React web interface
- Native Android mobile application using Retrofit for backend API calls
- Supabase PostgreSQL database integration
- Vercel deployment for the web frontend
- Render deployment for the Spring Boot backend
- Google sign-in for regular user accounts on web and mobile

1.0 INTRODUCTION

1.1 Purpose

This document serves as the design specification for the RentEasy system. It provides the functional and non-functional requirements, system architecture, API contracts, database design, technology stack, and implementation details needed to guide development and ensure that the web, mobile, backend, payment, and database components work together properly.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: RentEasy
Domain: E-commerce/Rental Services

Primary Users:

1. Customers
2. Administrators

Problem Statement: Users need a simple way to browse, list, and rent items without using a large or complex e-commerce platform.

Solution:

RentEasy provides a rental marketplace where customers can register, log in, browse approved rental listings, view product details, add items to a rental cart, set rental days, and proceed to PayMongo checkout. Owners can submit listings for approval, while administrators can manage products and review pending listings.

2.2 Core User Journeys

Journey 1: First-time Customer Rental

1. The user visits the web or mobile application.
2. The user opens the registration page.
3. The user creates an account using first name, last name, email, and password.
4. The user logs in using valid credentials.
5. The user browses approved rental listings.
6. The user selects an item to view product details.
7. The user adds the item to the rental cart.
8. The user opens the cart and updates the number of rental days if needed.
9. The user proceeds to checkout.
10. The user enters delivery/contact information.
11. The user starts PayMongo checkout.

12. The user receives an order confirmation after successful payment return.

Journey 2: Returning Customer Rental

1. The user logs in with existing credentials.
2. The user browses or searches rental listings.
3. The user adds an item to the cart.
4. The user updates rental days or removes cart items.
5. The user proceeds to checkout.
6. The user completes PayMongo checkout.
7. The user can view order history from the profile page.

Journey 3: Owner Listing Submission

1. The user logs in as a regular user.
2. The user goes to the listing creation page.
3. The user enters product name, category, price, stock, image, and description.
4. The user submits the listing.
5. The listing is saved with pending status.
6. The listing appears in the admin pending approval section.
7. Once approved by an admin, the listing appears in the customer catalog.

Journey 4: Administrator Product Review and Management

1. The admin logs in using admin credentials.
2. The admin opens the admin dashboard.
3. The admin views product metrics, products, pending approvals, orders, and users.
4. The admin approves or rejects pending product listings.
5. The admin views product details within the admin section.
6. The admin removes products when needed.

2.3 Feature List (MoSCoW)

MUST HAVE

1. User registration
2. User login
3. Frontend logout
4. JWT-based authentication for protected endpoints
5. Approved product catalog
6. Product detail view
7. Product search by name/category on the frontend
8. Product listing submission

9. Admin product approval and rejection
10. Admin product deletion
11. Shopping cart: add item, view cart, update rental days, remove item
12. Checkout form with delivery/contact details
13. PayMongo checkout session creation
14. Order confirmation page
15. User profile page
16. Profile picture change on the web frontend
17. Admin dashboard
18. Mobile app login/register and backend API communication through Retrofit
19. Backend-persisted orders table
20. Google sign-in for regular users

SHOULD HAVE

1. Responsive web design for desktop and mobile screen sizes
2. Loading indicators during login, registration, catalog loading, cart loading, listing submission, and checkout
3. Clear error messages when backend actions fail
4. Order history display in the user profile
5. Admin order/status display based on stored order records
6. Product categories
7. Separate admin table from regular users
8. Backend profile update endpoint
9. Backend order management endpoint

COULD HAVE

1. Backend profile image upload endpoint
2. PayMongo webhook verification
3. Product edit form for existing listings
4. Advanced dashboard charts
5. Wishlist or saved items
6. Email notifications

WON'T HAVE

1. Forgot password flow
2. Multi-language support
3. Advanced analytics
4. Real-time chat
5. Backend search endpoint

6. Supabase Storage file upload integration
7. Third-party login for admin accounts

2.4 Detailed Feature Specifications Feature:

User Authentication

- **Screens:** Registration, Login
- **Fields:** First Name, Last Name, Email, Password, Confirm Password
- **Validation:** Email format, password strength, uniqueness
    - Required fields must be filled in.
    - Password and confirm password must match during registration.
    - Duplicate email registration is rejected by the backend.
    - Invalid login credentials show an error message.
- API Endpoints:
    - POST /api/auth/register
    - POST /api/auth/login
    - POST /api/auth/google
- Security:
    - Passwords are hashed using BCrypt.
    - Successful login returns a JWT token.
    - Protected backend endpoints require a bearer token.
    - Logout is handled on the frontend by removing the stored token.
    - Google sign-in uses a Google ID token from the web or mobile client. The backend verifies the token against the configured Google OAuth client ID before issuing a RentEasy JWT token.
      Note: Google sign-in is available only for regular users. Admin accounts still use email/password login.

Feature: Rental Catalog

- **Screens:** Product Listing, Product Detail
- **Display:** Grid layout, product image, name, category, rental price per day, stock/status information, and product details.
- **Search:** Search is handled on the frontend using the catalog data already loaded from the backend.
- **API Endpoints:**
    - GET /api/products
    - GET /api/products/all-approved
    - GET /api/products/pending
- **Admin Functions:**
    - PUT /api/products/{id}/status

- DELETE /api/products/{id}

Feature: Product Listing Submission

- **Screens:** Create Listing, My Listings
- **Fields:** Product Name, Category, Price, Stock, Image/Image URL, Description
- **Process:**
  - A regular user submits a product listing.
  - The listing is saved with pending status.
  - Admin reviews the listing.
  - Approved listings appear in the public catalog.
  - The backend assigns the listing owner using the authenticated JWT token email. The front end no longer decides the product owner.
- **API Endpoint:**
  - POST /api/products

Feature: Shopping Cart

- **Screens:** Cart View
- **Functions:**
  - Add product to cart
  - View cart items
  - Update rental days
  - Remove cart item
  - Calculate subtotal, service fee, and total
- **Persistence:** Cart items are stored in the database per user.
- **API Endpoints**
  - GET /api/cart?email=<userEmail>
  - POST /api/cart/add
  - PUT /api/cart/{cartItemId}/days
  - DELETE /api/cart/{cartItemId}
  - GET /api/products/mine

Feature: Checkout and Payment

- **Screens:** Checkout Form, Order Confirmation
- **Data Collected:** Full Name, Email, Phone, Address, City, ZIP Code
- **Process:**
  - User reviews cart items.
  - User enters delivery/contact information.
  - System creates a PayMongo checkout session.
  - User is redirected to PayMongo checkout.
  - After payment return, the app displays order confirmation.
  - Cart items are cleared after checkout is started.

- **API Endpoint:**
    - POST /api/payments/paymongo/checkout

Feature: User Profile

- **Screens:** User Profile
- **Functions:**
    - View profile information
    - Update profile name and phone number
    - Change/remove profile picture on the frontend
    - View rental order history
- **Persistence:** User profile information such as first name, last name, email, and phone number is stored in the backend users table. Profile updates are handled through the backend profile endpoint. Profile picture changes are handled on the frontend only in the current implementation. Rental order history is retrieved from the backend rental_orders and rental_order_items tables.
- API Endpoints:
    - GET /api/user/profile
    - PUT /api/user/profile
    - GET /api/orders/my

Feature: Admin Panel

- **Screens:** Admin Dashboard, Products, Pending Approval, Orders, Users, Admin Product Details
- **Functions:**
    - View dashboard metrics
    - View all products
    - View product details
    - Remove products
    - View pending listings
    - Approve pending listings
    - Reject pending listings
    - View backend rental order records from the rental_orders and rental_order_items tables.
    - View admin/user summary information
- **Access Control:** Admin access is separated from regular user access. Admin accounts are stored in a separate admins table.
- **API Endpoint:**
    - GET /api/products
    - GET /api/products/pending
    - PUT /api/products/{id}/status

- DELETE /api/products/{id}
- GET /api/orders
- PUT /api/orders/{orderNumber}/status

2.5 Acceptance Criteria

AC-1: Successful User Registration

- ➢ Given I am a new user
- ➢ When I enter valid first name, last name, email, password, and matching confirm password
- ➢ And I click “Create Account”
- ➢ Then my account should be created
- ➢ And I should be redirected to the login page
- ➢ And duplicate emails should be rejected

AC-2: User Login

- ➢ Given I am a registered user
- ➢ When I enter correct credentials
- ➢ Then I should receive a JWT token
- ➢ And I should be redirected to the customer home page
- ➢ And invalid credentials should show a clear error message

AC-3: Admin Login

- ➢ Given I am an administrator
- ➢ When I enter valid admin credentials
- ➢ Then I should receive a JWT token
- ➢ And I should be redirected to the admin dashboard

AC-4: Rental Catalog Browsing

- ➢ Given I am logged in as a customer
- ➢ When I open the catalog page
- ➢ Then I should see approved listings from the database
- ➢ And I should be able to search listings by visible product information

AC-5: Product Detail Viewing

- ➢ Given I am viewing the catalog
- ➢ When I select a product
- ➢ Then I should see product details, price, category, image, description, and owner-related information when available

AC-6: Shopping Cart Management

- Given I am logged in as a customer
- When I add a product to my cart
- Then the item should appear in the cart
- And I can update rental days
- And I can remove the cart item
- And totals should update based on price and rental days

AC-7: Checkout Flow

- Given I have cart items
- When I proceed to checkout
- And I enter valid delivery/contact information
- And I click Pay with PayMongo
- Then the backend should create a PayMongo checkout session
- And I should be redirected to the PayMongo checkout URL when available

AC-8: Order Confirmation

- Given I return from a successful PayMongo checkout
- When the order confirmation page loads
- Then I should see payment/order confirmation details
- And the rental order should be saved in the backend rental_orders table
- And the ordered items should be saved in the backend rental_order_items table

AC-9: Listing Submission

- Given I am logged in as a regular user
- When I submit a product listing with valid details
- Then the listing should be saved
- And it should be marked as pending until admin approval

AC-10: Admin Listing Approval

- Given I am logged in as an admin
- When I approve a pending listing
- Then the listing status should become approved
- And it should appear in the customer catalog

AC-11: Admin Listing Rejection

- Given I am logged in as an admin
- When I reject a pending listing
- Then the listing status should be updated as rejected
- And it should not appear in the approved customer catalog

AC-12: Admin Product Removal

- Given I am logged in as an admin
- When I remove a product
- Then the product should be deleted from the database
- And related cart items should be removed

AC-13: Profile Management

- Given I am logged in
- When I update profile fields or change the profile picture
- Then the updated name and phone number should be saved through the backend profile endpoint
- And profile picture changes should reflect on the frontend

AC-14: Security and Authentication

- Given I access a protected endpoint
- When I do not provide a valid JWT token
- Then the backend should deny access with an unauthorized response

AC-15: Mobile API Communication

- Given I use the Android mobile app on an emulator
- When the backend is running locally
- Then the app should communicate with the backend through Retrofit using http://10.0.2.2:8080/

AC-16: Google Sign-In

- Given I am on the login page
- When I select Continue with Google using a valid Google account
- Then the backend should verify the Google ID token
- And create or identify my regular user account in the users table
- And return a RentEasy JWT token
- And redirect me to the customer homepage
- And my submitted listings should be linked to my Google account.

AC-17: Owner Listings

- Given I am logged in as a regular user
- When I open My Listings
- Then the system should request my products from /api/products/mine
- And only products owned by my authenticated account should be displayed.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- The system uses lightweight REST API endpoints with JSON responses for authentication, product listings, cart operations, listing approval, and PayMongo checkout creation.
- The web frontend is built with React and Vite, allowing faster development builds and optimized production builds.
- The product catalog, cart, profile, listings, checkout, and admin dashboard pages display loading states while data is being fetched.
- The mobile app uses Retrofit with Kotlin Coroutines so API calls run asynchronously and do not block the main UI thread.
- Product images are handled through imageUrl values and frontend image preview behavior. The current backend does not implement a dedicated file upload or download service.
- No formal load testing has been performed, so exact guarantees for concurrent users, response time, or high-load performance are not claimed.

3.2 Security Requirements

- Protected backend endpoints require JWT bearer token authentication through the Authorization header.
- User and admin passwords are stored using BCrypt password hashing.
- JWT tokens expire after 60 minutes.
- The backend uses Spring Security with stateless session management.
- Public access is allowed for authentication endpoints and approved product browsing, while cart, product creation, product deletion, product approval, and payment checkout require authentication.
- The web frontend stores the JWT token in browser localStorage after login and removes it during logout.
- CORS is configured for the local Vite frontend and the configured deployed frontend URL.
- The deployed frontend and backend use HTTPS through Vercel and Render. The Android emulator uses local HTTP through http://10.0.2.2:8080/ during development.
- SQL injection risk is reduced by using Spring Data JPA/Hibernate instead of manual SQL string building.
- CSRF protection is disabled because the backend is a stateless JWT-based REST API.
- Google ID tokens are verified by the backend before a RentEasy JWT token is issued.
- Google sign-in is restricted to regular users and cannot be used for admin login.

3.3 Compatibility Requirements

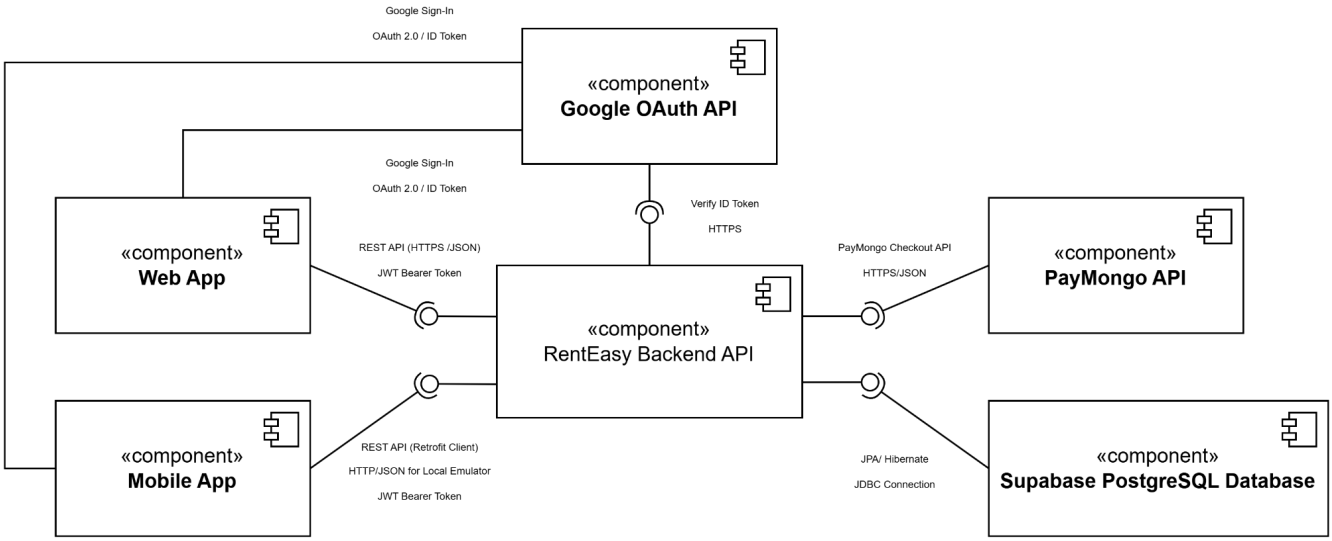
- The web application supports modern browsers such as Chrome, Edge, Firefox, and Safari.
- The web interface is responsive and uses Tailwind CSS utility classes for desktop, tablet, and mobile screen layouts.
- The Android mobile app supports Android API Level 24 and above.
- The mobile app communicates with the backend using Retrofit and JSON-based REST API calls.
- The backend runs on Java 21 and can be deployed to a Java-supported hosting environment such as Render.
- The database is hosted on Supabase PostgreSQL and accessed by the backend through PostgreSQL JDBC, JPA, and Hibernate.
- The frontend is deployed through Vercel, while the backend is deployed through Render.
- Mobile Google sign-in requires a Google OAuth Android client configured with the Android package name and SHA-1 certificate fingerprint.

3.4 Usability Requirements

- Users can register, log in, browse approved listings, view product details, add items to cart, update rental days, remove cart items, and proceed to checkout.
- Users can create product listings that are submitted for admin approval before appearing in the public catalog.
- Users can manage their profile information and update their profile picture locally in the web interface.
- Admin users can view dashboard metrics, view products, approve or reject pending listings, remove products, and view admin-side product details.
- The interface uses consistent navigation, buttons, forms, and card layouts across the main user pages.
- Error messages are shown for failed login, failed registration, cart loading issues, listing removal issues, backend connection issues, and PayMongo checkout errors.
- Loading indicators are shown during major actions such as login, registration, product loading, cart loading, listing submission, and checkout.
- The system uses clear form placeholders such as “Enter your email address,” “Enter your password,” and “Enter your first name” to guide user input.
- The checkout flow provides a PayMongo payment button and redirects users to the PayMongo checkout session when available.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 21, Spring Boot 3.5.0, Spring Web, Spring Security, Spring Data JPA, Hibernate, PostgreSQL JDBC Driver, JJWT for JWT authentication, Google ID token verification, and Maven.
- **Database:** Supabase PostgreSQL, accessed by the Spring Boot backend through Spring Data JPA, Hibernate, and JDBC. The database stores users, admins, products, cart items, rental orders, rental order items, product images as text URLs/base64 strings, and profile image data through avatar_url.
- **Web Frontend:** React 19, Vite, JavaScript/JSX, React Router, Tailwind CSS, Lucide React icons, Google Identity Services, npm, and browser localStorage for storing frontend session data such as the JWT token and local profile state.
- **Mobile App:** Native Android application using Kotlin, Retrofit 2.9.0 for REST API calls, Gson Converter for JSON parsing, Kotlin Coroutines for asynchronous operations, AndroidX Lifecycle, AppCompat, Material Components, Google Play Services Auth for Google sign-in, and Gradle for building the APK.
- **API Communication:** REST API using JSON request and response bodies. Protected endpoints use JWT bearer tokens in the Authorization header to identify the logged-in user and secure customer/admin actions.
- **Authentication:** Email/password authentication, Google Authentication using Google ID tokens, backend token verification, and JWT generation for authenticated RentEasy sessions.

- **Payments:** PayMongo Checkout API for creating payment checkout sessions and redirecting users to complete rental payment.
- **Build Tools:** Maven for the Spring Boot backend, npm/Vite for the React web frontend, and Gradle for the Android mobile application.
- **Deployment:** Web frontend deployed on Vercel, Spring Boot backend deployed on Render, database hosted on Supabase PostgreSQL, and the mobile application distributed and tested as an APK.

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	Local Development: http://localhost:8080 Production: https://it342-nuevas-renteasy-1.onrender.com
Format	Most requests and responses use JSON. Some endpoints return plain text or an empty response body.
Authentication	Protected endpoints require a JWT bearer token in the Authorization header. Google login uses a Google ID token sent to the backend. After verification, the backend returns the application JWT token used for protected RentEasy endpoints.
Authorization	Bearer <token>

Response Structure	The backend does not use one universal response wrapper for every endpoint. Responses vary depending on the endpoint.
	
	Example product response:
	{
	"success": true,
	"product": {}
	}
	
	Example PayMongo checkout response:
	{
	"checkoutUrl":
	"https://checkout.paymongo.com/...",
	"sessionId": "cs_...",
	"referenceNumber": "RE-2026-12345"
	}
	
	Example registration response:

	User registered successfully

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	Creates a new regular user account.
API URL	/api/auth/register
HTTP Request Method	POST
Authentication	None

Request Payload	{
	"firstName": "Josh",
	"lastName": "Nuevas",
	"email": "josh@example.com",
	"phone": "09123456789",
	"password": "password123"
	}
	
	Response:
	User registered successfully
	or
	Email already exists

User/Admin Login

Description	Logs in either a regular user or an admin. The backend checks the users table first, then the admins table.
API URL	/api/auth/login
HTTP Method	POST
Authentication	None

Request Payload	{
	"email": "josh@example.com",
	"password": "password123"
	}
	
	Success Response:

	<jwt-token> Error Response: Invalid credentials Notes: There is no backend logout endpoint. Logout is handled on the frontend by removing the saved token from local storage

Google Login

Description	Authenticates a regular user using a Google ID token.
API URL	/api/auth/google
HTTP Method	POST
Authentication	None

Request Payload	{
	"idToken": "google_id_token"
	}
	
	Success Response:
	<jwt-token>
	
	Error Response:
	{
	"detail": "Invalid Google ID token"
	}
	
	Notes:
	The backend verifies the Google ID token against the configured Google client ID. If the Google account does not yet exist in the users table, the backend creates a regular user account. Google sign-in is not used for admin accounts.

Product Endpoints

Get All Products

Description	Returns all products, including approved and pending products.
API URL	/api/products
HTTP Method	GET
Authentication	Not required for GET requests.
Request Payload	None

Response Structure	[
	{
	"productId": 1,
	"name": "Sony Camera",
	"description": "Camera rental",
	"price": 100,
	"stock": 1,
	"category": "Cameras",
	"imageUrl": "https://...",
	"status": "APPROVED",
	"owner": {
	"userID": 1,
	"email": "josh@example.com",
	"firstName": "Josh",
	"lastName": "Nuevas",
	"phone": "09123456789"
	},
	"createdAt": "2026-05-19T10:30:00"
	}
	]

Get Approved Products

Description	Returns only products approved for display in the catalog.
API URL	/api/products/all-approved
HTTP Method	GET
Authentication	Not required for GET requests.
Response Structure	[{

	"productId": 1, "name": "Sony Camera", "description": "Camera rental", "price": 100, "stock": 1, "category": "Cameras", "imageUrl": "https://...", "status": "APPROVED" }]

Get Pending Products

Description	Returns products waiting for admin approval.
API URL	/api/products/pending
HTTP Method	GET
Authentication	JWT used by the frontend admin page

Response Structure	[
	{
	"productId": 2,
	"name": "Basketball",
	"description": "Outdoor ball",
	"price": 100,
	"stock": 1,
	"category": "Outdoor",
	"imageUrl": "https://...",
	"status": "PENDING"
	}
	]

Create Product Listing

Description	Creates a new product listing. New listings start as pending unless approved later by the admin.
API URL	/api/products
HTTP Method	POST
Authentication	Required
Request Payload	{ "name": "Sony Camera",

	"description": "Camera rental", "price": 100, "stock": 1, "category": "Cameras", "imageUrl": "https://...", } Notes: The backend determines the owner from the authenticated JWT token, not from the request payload.

Response Structure	{
	"success": true,
	"product": {
	"productId": 1,
	"name": "Sony Camera",
	"status": "PENDING"
	}
	}

Update Product Status

Description	Used by admin to approve or reject a listing.
API URL	/api/products/{id}/status
HTTP Method	PUT
Authentication	Required

Request Payload	{
	"status": "APPROVED"
	}
	or
	{
	"status": "REJECTED"
	}

Response Structure	{
	"success": true
	}

Delete Product

Description	Deletes a product listing and removes related cart items.
API URL	/api/products/{id}
HTTP Method	DELETE
Authentication	Required

Response Structure	{
	"success": true
	}

Get My Products

Description	Returns product listings owned by the currently logged-in user.
API URL	/api/products/mine
HTTP Method	GET
Authentication	Required

Response Structure	[
	{
	"productId": 1,
	"name": "Sony Camera",
	"description": "Camera rental",
	"price": 100,
	"stock": 1,
	"category": "Cameras",
	"imageUrl": "https://...",
	"status": "APPROVED",
	"owner": {
	"userID": 1,
	"email": "josh@example.com"
	}
	}
	]

Cart Endpoints

Get User Cart

Description	Returns the logged-in user's cart items.
API URL	/api/cart?email=<userEmail>
HTTP Method	GET
Authentication	Required

Response Structure	[
	{
	"id": 1,
	"days": 2,
	"userEmail": "josh@example.com",
	"product": {
	"productId": 1,
	"name": "Sony Camera",
	"price": 100,
	"imageUrl": "https://..."
	}
	}
	]

Add Item to Cart

Description	Adds a product to the user's cart. If the product already exists in the cart, the rental days increase by 1
API URL	/api/cart/add
HTTP Method	POST
Authentication	Required

Request Payload	{
	"productId": 1,
	"userEmail": "josh@example.com"
	}

Response Structure	200 OK No response body.

Update Cart Item Days

Description	Updates the number of rental days for a cart item.
API URL	/api/cart/{cartItemId}/days
HTTP Method	PUT
Authentication	Required

Request Payload	{
	"days": 3
	}

Response Structure	200 OK No response body.

Remove Item from Cart

Description	Removes an item from the cart.
API URL	/api/cart/{cartItemId}
HTTP Method	DELETE
Authentication	Required
Response Structure	200 OK No response body.

Notes: *The response field "id" represents the cart_item_id primary key in the database.*

Order Endpoints

Create Order

Description	Creates a rental order record after checkout.
API URL	/api/orders
HTTP Method	POST
Authentication	Required

Request Payload	{
	"orderNumber": "RE-2026-12345",
	"delivery": {
	"name": "Josh Anton Nuevas",
	"email": "josh@example.com",
	"phone": "09123456789",
	"address": "Cebu City",
	"city": "Cebu City",

	"zip": "6000"
	},
	"subtotal": 200,
	"serviceFee": 10,
	"total": 210,
	"items": [
	{
	"productId": 1,
	"name": "Sony Camera",
	"description": "Camera rental",
	"price": 100,
	"days": 2,
	"imageUrl": "https://..."
	}
	]
	}

Response Structure	{
	"orderNumber": "RE-2026-12345",
	"subtotal": 200,
	"serviceFee": 10,
	"total": 210,
	"status": "Processing",
	"customerEmail": "josh@example.com",
	"createdAt": "2026-05-20T10:30:00",
	"items": []
	}

Get My Orders

Description	Returns the logged-in user's rental orders.
API URL	/api/orders/my
HTTP Method	GET
Authentication	Required
Response Structure	List of rental orders for the logged-in user.

Get All Orders

Description	Returns all rental orders for admin viewing.
API URL	/api/orders
HTTP Method	GET

Authentication	Required
Response Structure	List of all rental orders.

Update Order Status

Description	Updates the status of an order.
API URL	/api/orders/{orderNumber}/status
HTTP Method	PUT
Authentication	Required

Response Payload	{
	"status": "Processing"
	}

Response Structure	Updated rental order details.

User Profile Endpoints

Get User Profile

Description	Returns the logged-in user's profile information.
API URL	/api/user/profile
HTTP Method	GET
Authentication	Required

Response Structure	{
	"userID": 1,
	"email": "josh@example.com",
	"firstName": "Josh",
	"lastName": "Nuevas",
	"phone": "09123456789",
	"createdAt": "2026-05-20T10:30:00"
	}

Update User Profile

Description	Updates the logged-in user's name and phone number.
API URL	/api/user/profile
HTTP Method	PUT
Authentication	Required

Response Payload	{
	"firstName": "Josh",
	"lastName": "Nuevas",
	"phone": "09123456789"
	}

Response Structure	Updated user profile details.

Payment Endpoint

Create PayMongo Checkout

Description	Creates a PayMongo checkout session for the selected cart items.
API URL	/api/payments/paymongo/checkout
HTTP Method	POST
Authentication	Required

Request Payload	{
	"orderNumber": "RE-2026-12345",
	"delivery": {
	"name": "Josh Anton Nuevas",
	"email": "josh@example.com",
	"phone": "09123456789",
	"address": "Cebu City",
	"city": "Cebu City",
	"zip": "6000"
	},
	"subtotal": 200,
	"serviceFee": 10,
	"total": 210,
	"items": [
	{
	"productId": 1,
	"name": "Sony Camera",
	"description": "Camera rental",
	"price": 100,
	"days": 2,
	"imageUrl": "https://..."
	}
	]

	} Optional Fields: **successUrl** - Custom redirect URL after successful PayMongo payment. **cancelUrl** - Custom redirect URL after cancelled PayMongo payment.

Response Structure	Success Response:
	{
	"checkoutUrl": "https://checkout.paymongo.com/...",
	"sessionId": "cs_...",
	"referenceNumber": "RE-2026-12345"
	}
	
	Error Response:
	{
	"detail": "PayMongo secret key is not configured."
	}

Mobile Payment Redirect Endpoints

PayMongo Mobile Success Redirect

Description	Redirects the mobile app back to the RentEasy app after successful PayMongo payment.
API URL	/api/payments/paymongo/mobile/success
HTTP Method	GET
Authentication	Not required

PayMongo Mobile Cancel Redirect

Description	Redirects the mobile app back to the RentEasy app after cancelling PayMongo payment.
API URL	/api/payments/paymongo/mobile/cancel
HTTP Method	GET
Authentication	Not required

5.3 Error Handling HTTP Status Codes

- 200 OK - Successful request
- 400 Bad Request - Invalid or incomplete request payload
- 401 Unauthorized - Missing, invalid, or expired JWT token
- 403 Forbidden - Request is authenticated but not allowed
- 404 Not Found - Requested product, cart item, order, or resource does not exist
- 500 Internal Server Error - Unexpected backend/server error
- 502 Bad Gateway - PayMongo checkout request failed
- 503 Service Unavailable - PayMongo secret key or Google client ID is missing/configuration is incomplete

Error Code Examples

Example 1	Invalid credentials Returned when the submitted email or password does not match a record in the users or admins table.

Example 2	{
	"detail": "PayMongo secret key is not configured."
	}
	
	Returned when the backend cannot start PayMongo checkout because the secret key is missing.

Example 3	{
	"detail": "PayMongo checkout failed: Invalid PayMongo checkout request."
	}
	
	Returned when PayMongo rejects the checkout request.

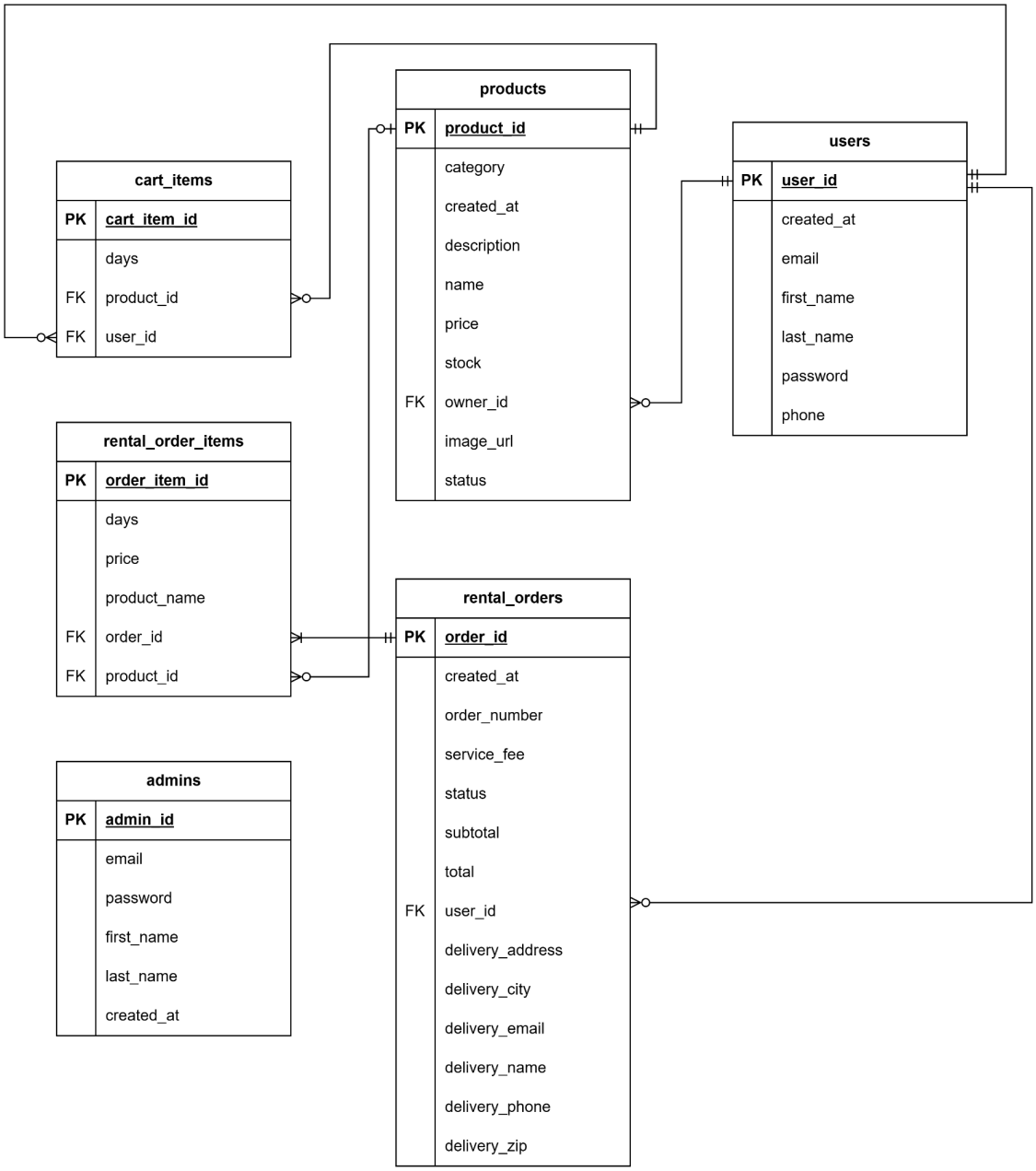
Example 4	404 Not Found Returned when a requested product, cart item, or rental order does not exist.

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Missing or expired JWT token
- AUTH-003: Unauthorized request
- VALID-001: Invalid or incomplete request data
- DB-001: Requested product, cart item, order, or user profile not found
- DB-002: Duplicate email or duplicate cart item
- PRODUCT-001: Product could not be created, updated, approved, rejected, or deleted
- CART-001: Cart item could not be added, updated, or removed
- ORDER-001: Rental order could not be created, viewed, or updated
- PAYMENT-001: PayMongo checkout failed
- PAYMENT-002: PayMongo secret key is missing
- SYSTEM-001: Internal server error
- AUTH-004: Invalid Google ID token
- AUTH-005: Google client ID is not configured
- AUTH-006: Google sign-in is not allowed for admin accounts

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **One-to-Many: User to Products:** A user can create zero or many rental product listings, while each product listing must belong to exactly one user.
- **One-to-Many: User to CartItems:** A user can have zero or many pending cart items, while each cart item must belong to exactly one user.
- **One-to-Many:** Product to CartItems: A product can appear in zero or many users' carts, while each cart item must reference exactly one product.
- **One-to-Many:** User to RentalOrders: A user can place zero or many rental orders, while each rental order must belong to exactly one user.
- **One-to-Many:** RentalOrder to RentalOrderItems: One rental order must have one or many rental order items, while each rental order item must belong to exactly one rental order.
- **One-to-Many:** Product to RentalOrderItems: A product can appear in zero or many rental order items, while each rental order item may reference one product. The product name and price are stored in the order item to preserve order history even if the product is later removed.
- **Separate Admin Accounts:** Admin accounts are stored separately from users because they are staff/system accounts used only for admin access and product approval management.

6.2 Key Tables

- **User:** Stores customer account information, authentication credentials, and contact details.
- **Admin:** Stores administrator account information separately from regular users.
- **Product:** Contains the rental item catalog, including pricing, stock, category, image URL, approval status, and owner information.
- **Cart_Item:** Stores the user's selected rental products and the number of rental days for each item.
- **Rental_Order:** Stores completed rental transactions, including totals, order status, customer delivery information, and the user who placed the order.
- **Rental_Order_Item:** Stores the individual products included in a rental order, including product name, price, and rental days.

Notes:
Google-authenticated users are stored in the same users table as regular users. The system does not use a separate Google users table because Google sign-in only changes the authentication method, not the account type.

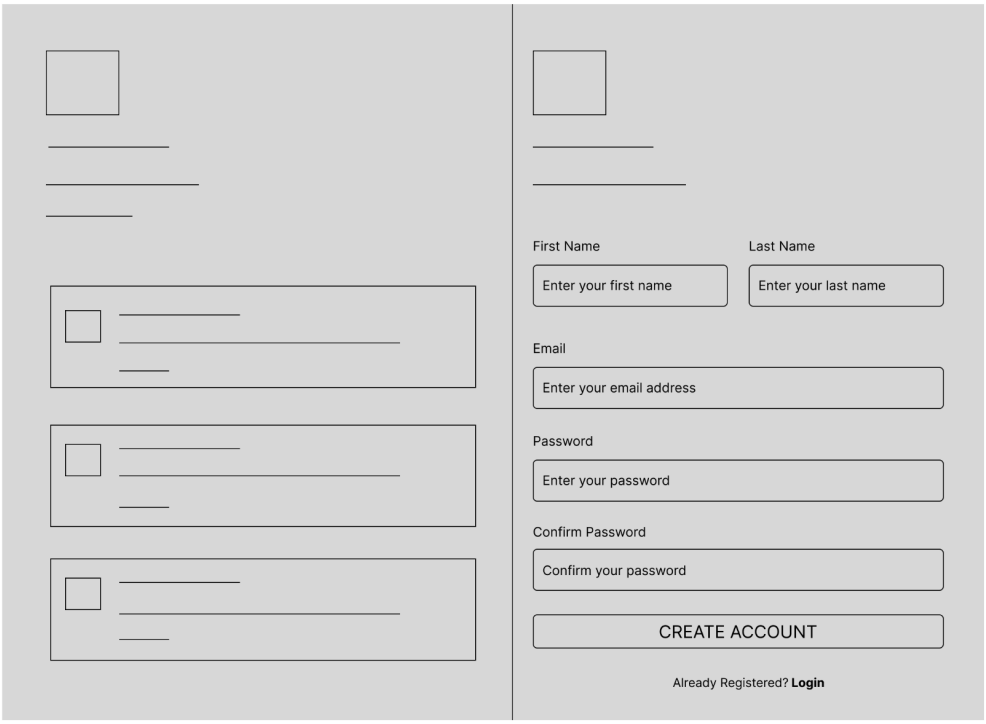
Table Structure Summary:

- **User:** user_id, email, password, first_name, last_name, phone, created_at
- **Admin:** admin_id, email, password, first_name, last_name, created_at
- Product: product_id, owner_id, name, description, price, stock, image_url, category, status, created_at
- **Cart_Item:** cart_item_id, user_id, product_id, days
- **Rental_Order:** order_id, order_number, user_id, subtotal, service_fee, total, status, delivery_name, delivery_email, delivery_phone, delivery_address, delivery_city, delivery_zip, created_at
- **Rental_Order_Item:** order_item_id, order_id, product_id, product_name, price, days

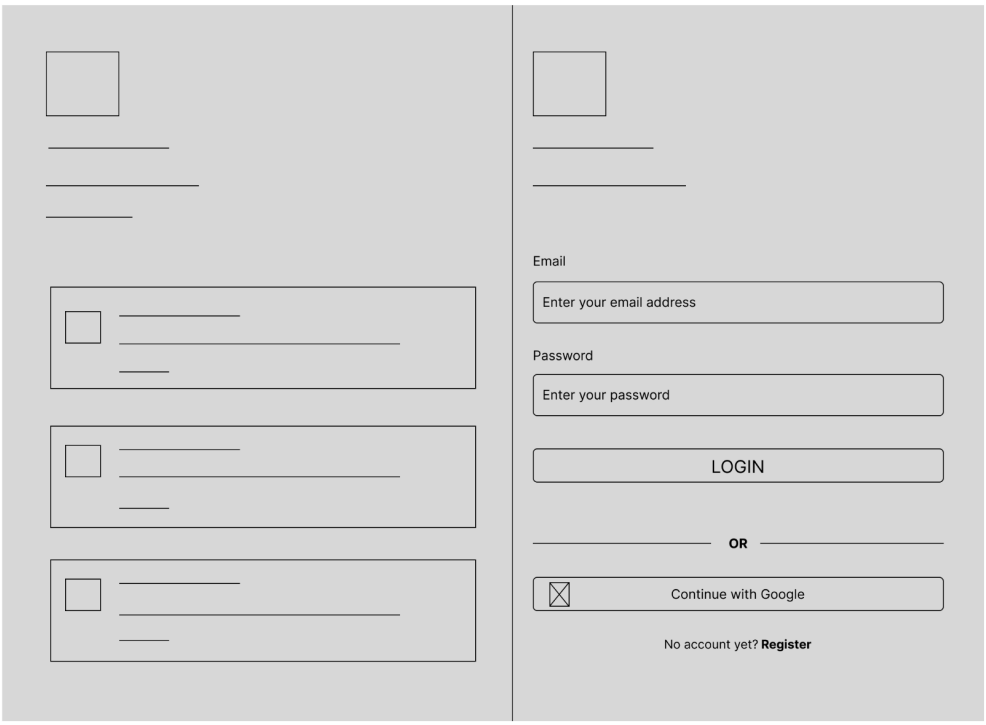
7.0 UI/UX DESIGN

7.1 Web Application Wireframes

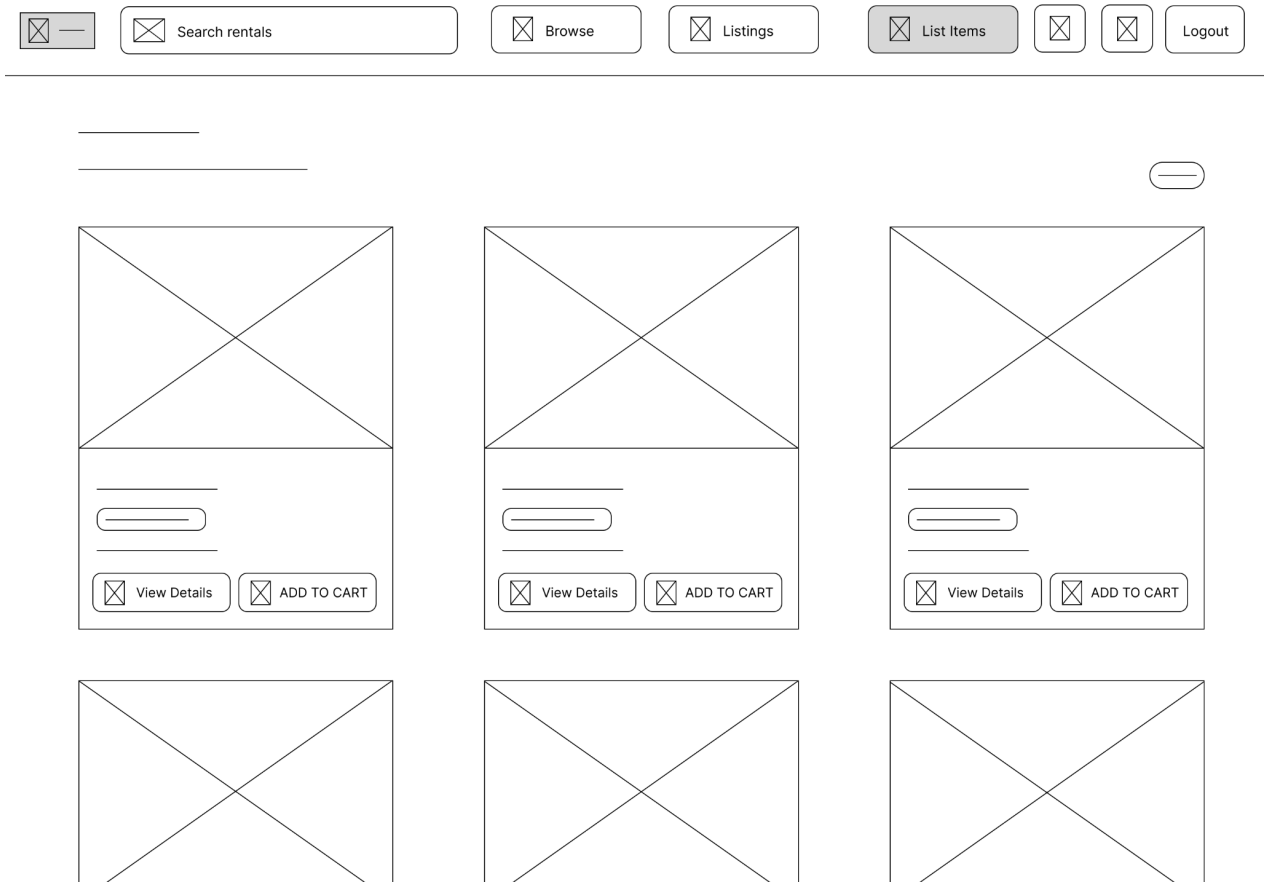
Register Page



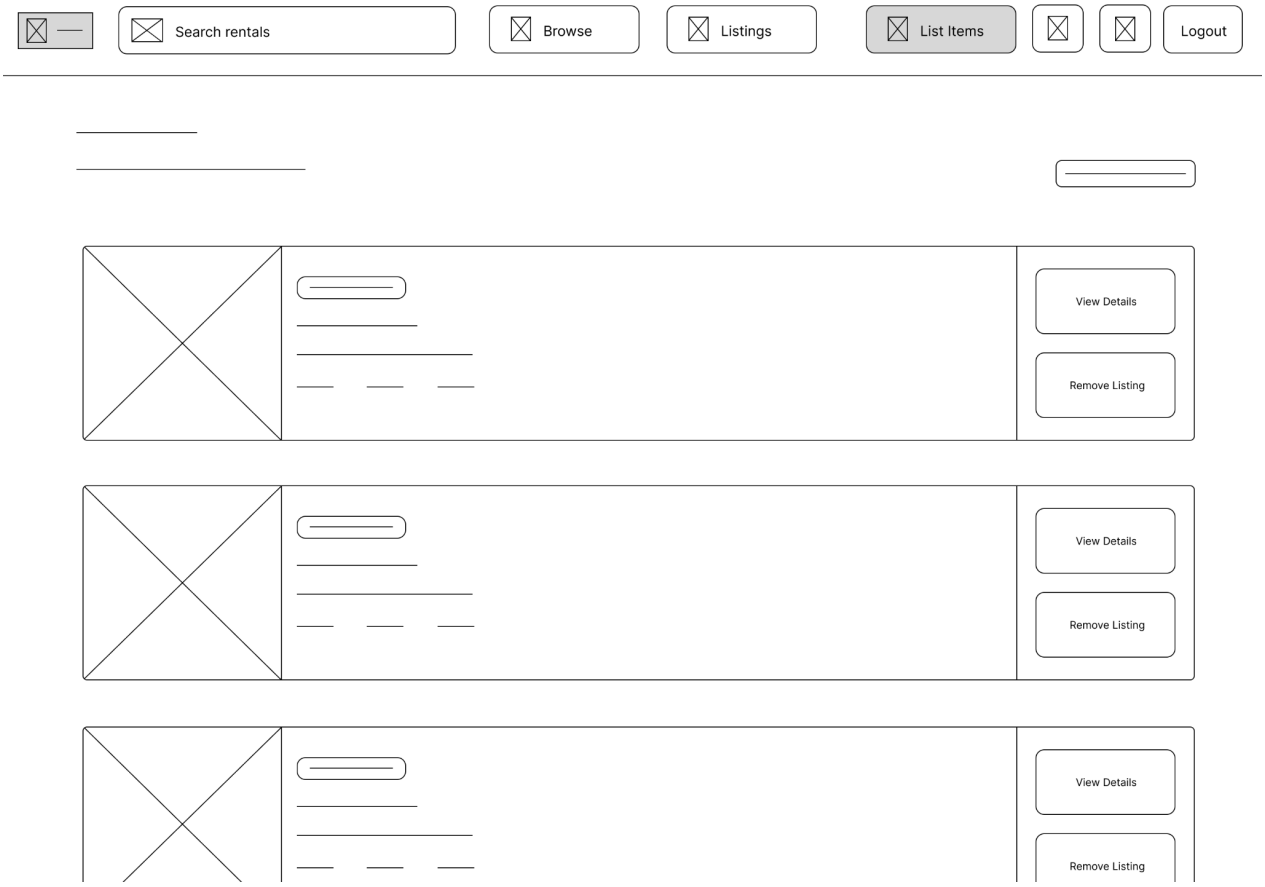
Login Page


Email
Enter your email address
Password
Enter your password
LOGIN
OR
Continue with Google
No account yet? Register

User Home Page



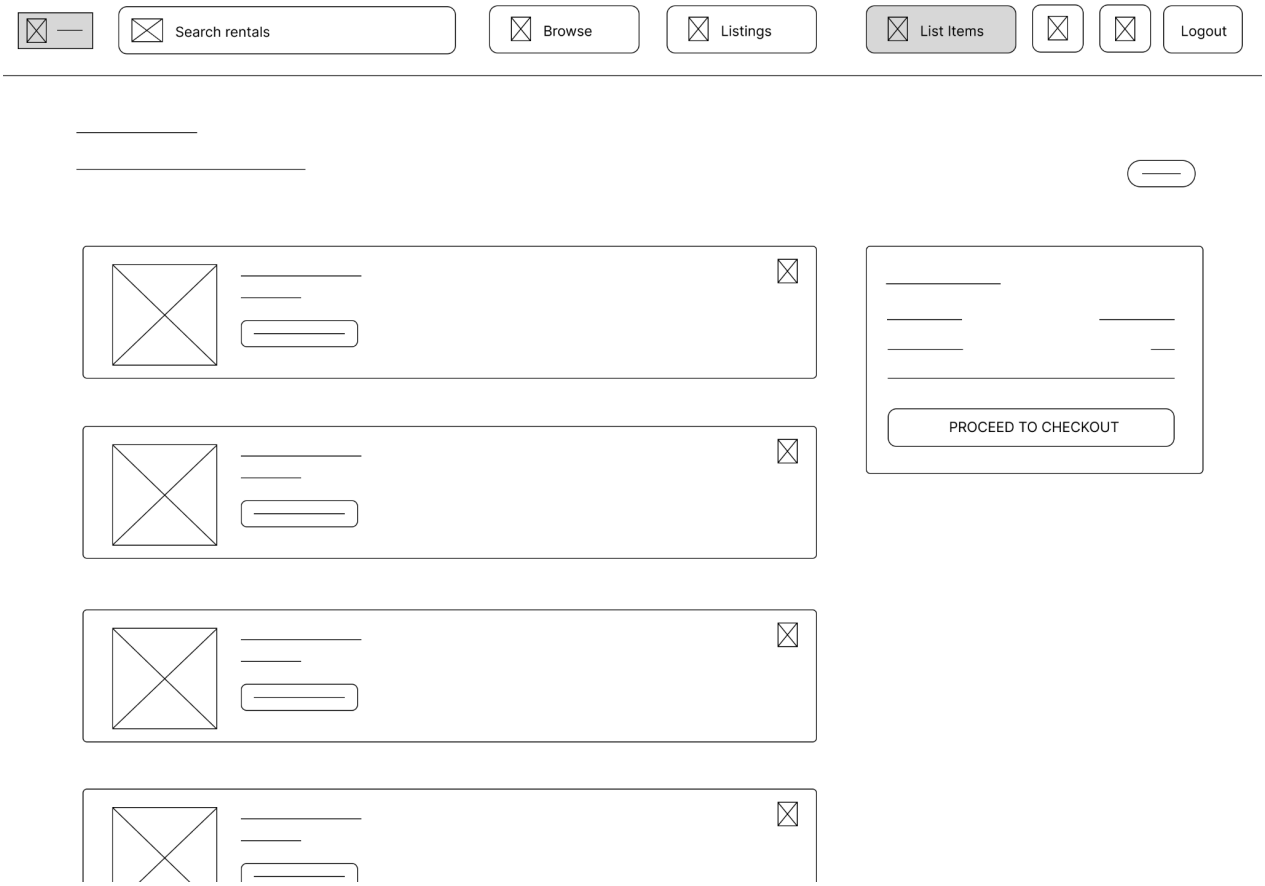
User Listings Page



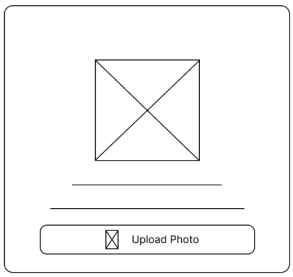
User List Items Page 1

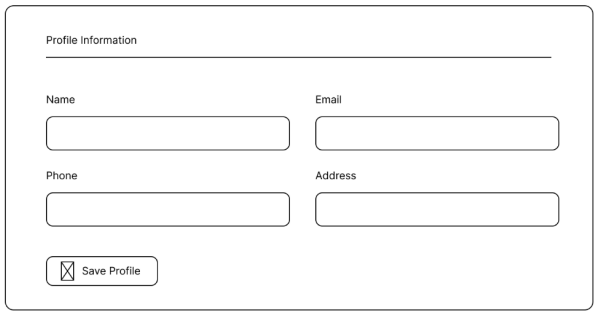
User List Items Page 2

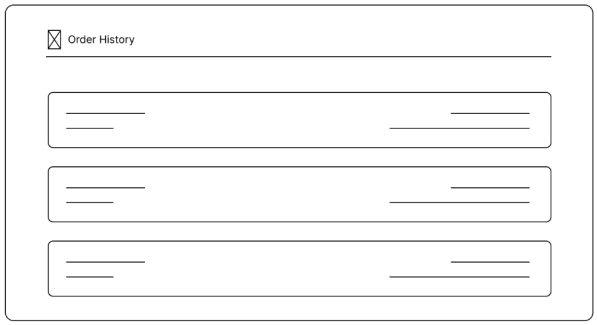
User Cart Page



User Profile Page

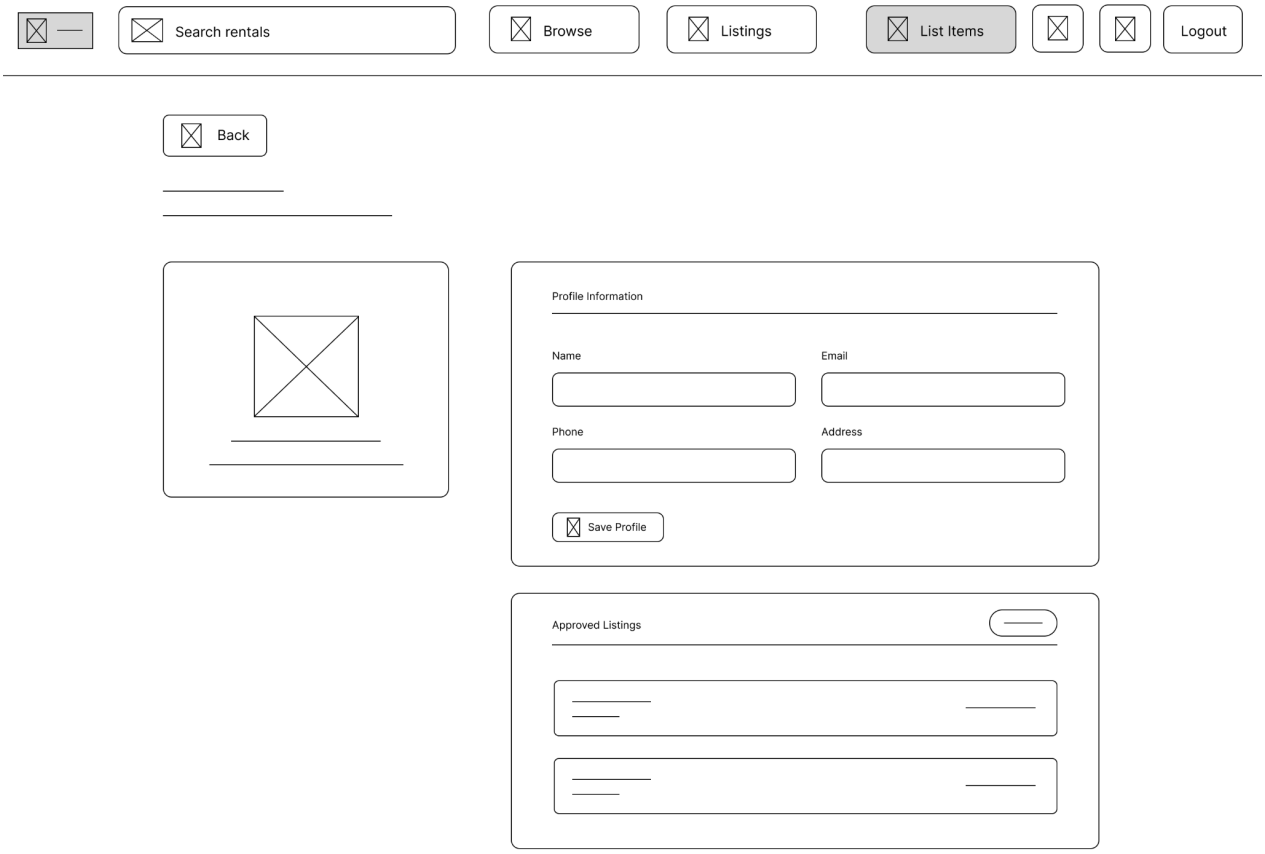




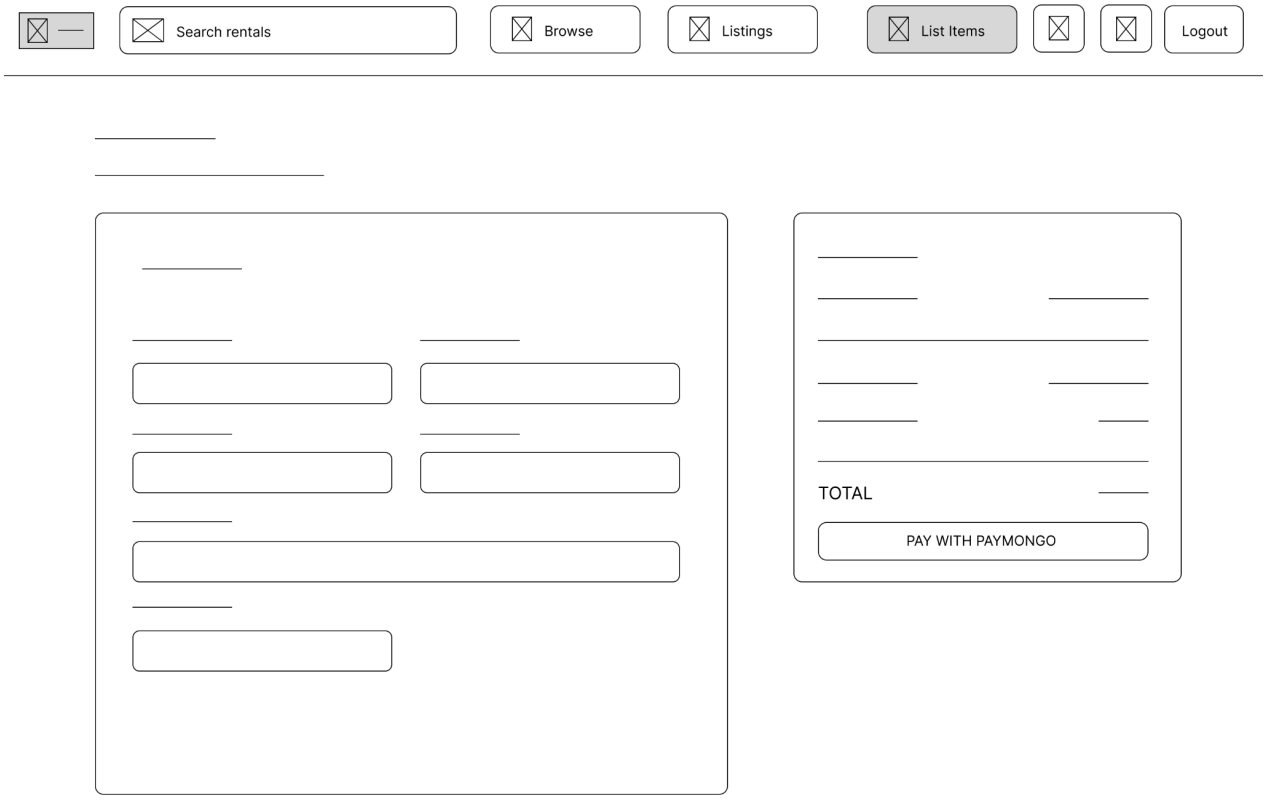


View Details Page

View Renter Profile Page

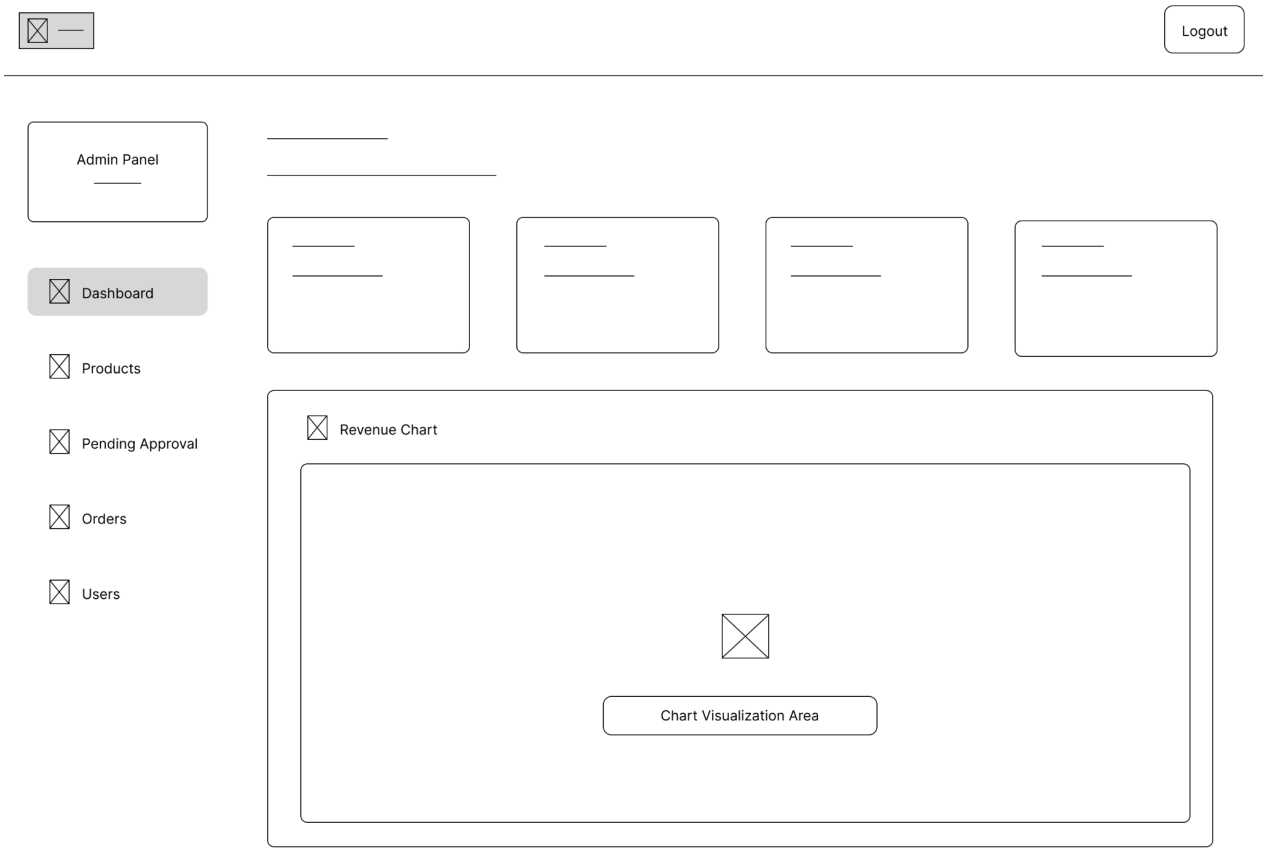


User Payment Page

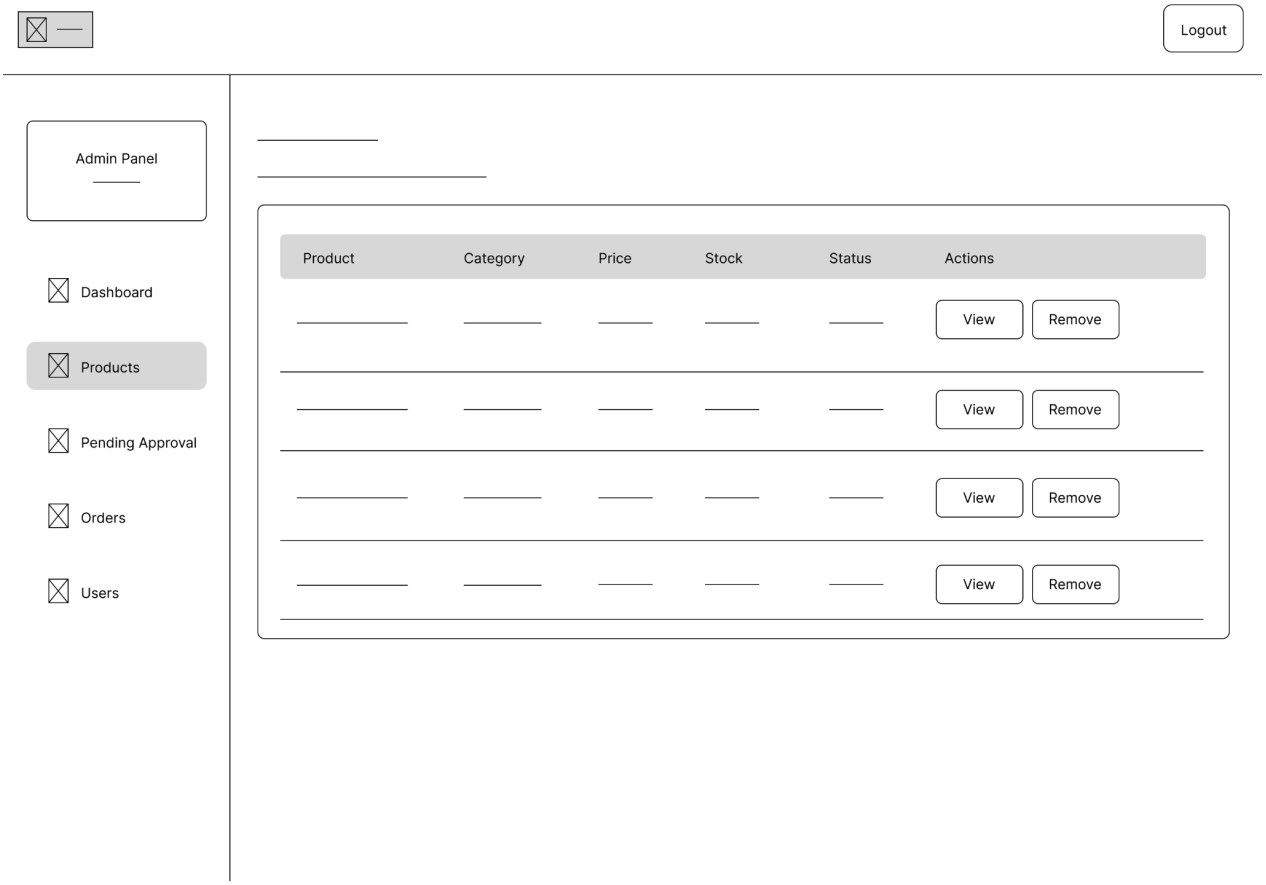


User Payment Received Page

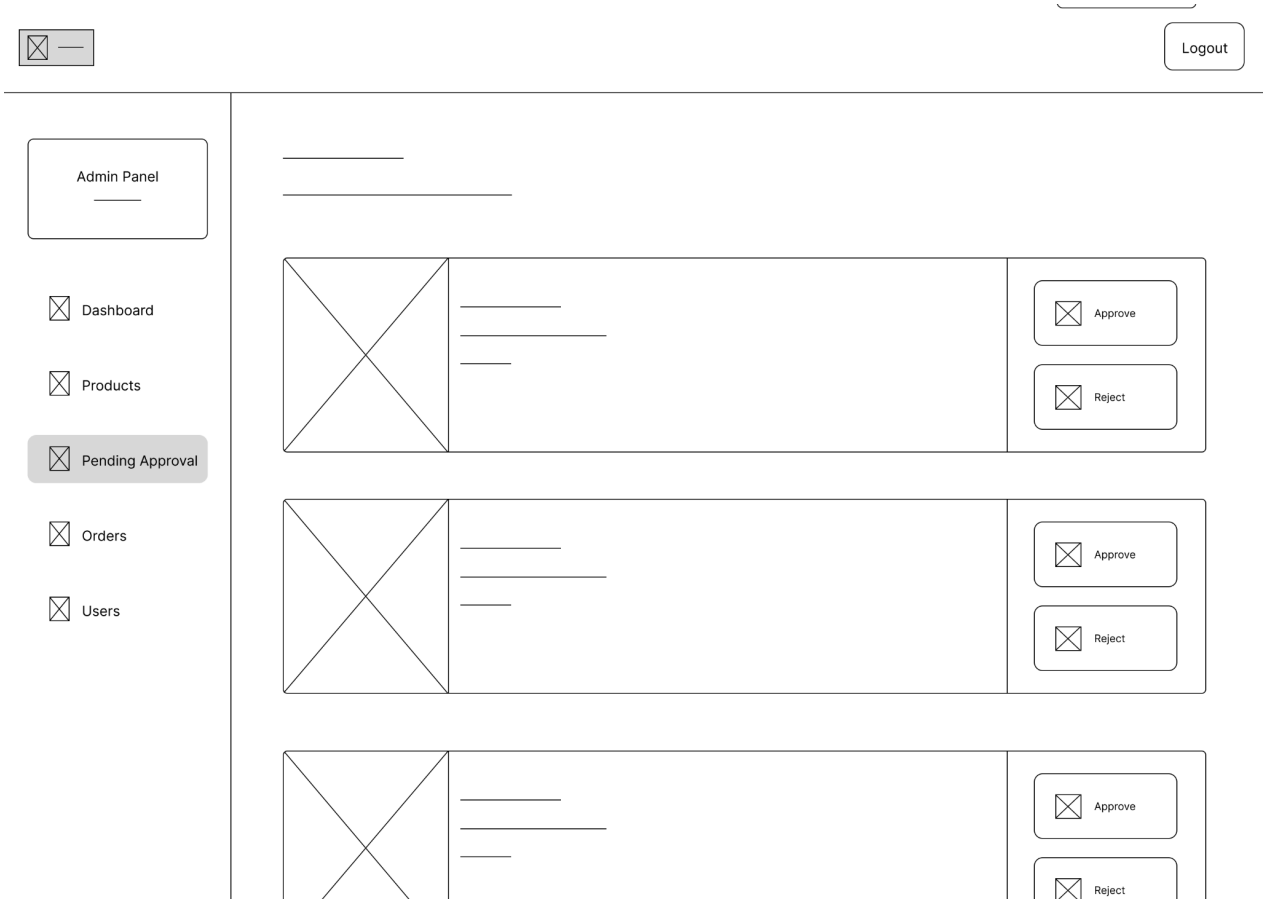
Admin Dashboard Page



Admin Product Management Page

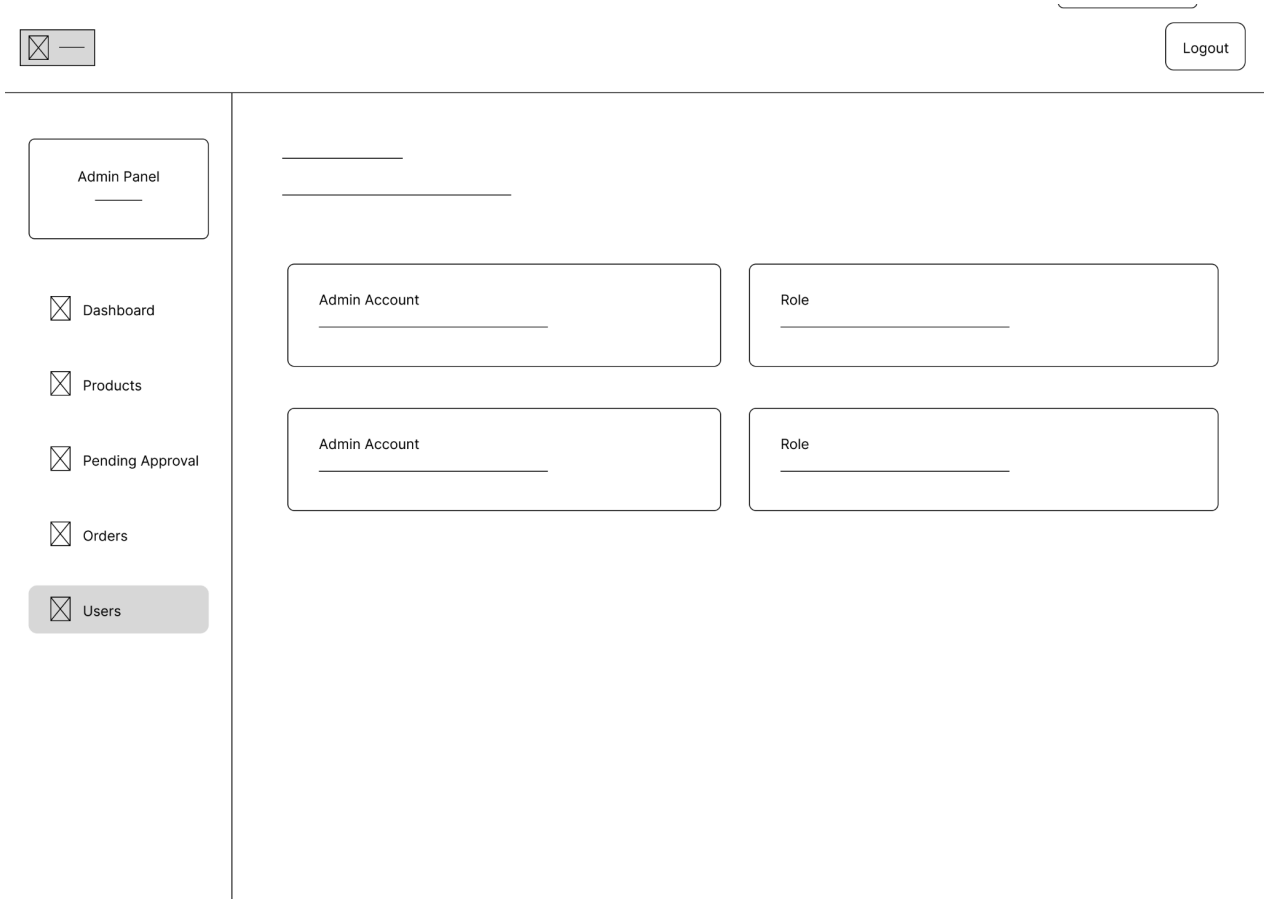


Admin Pending Approval Page

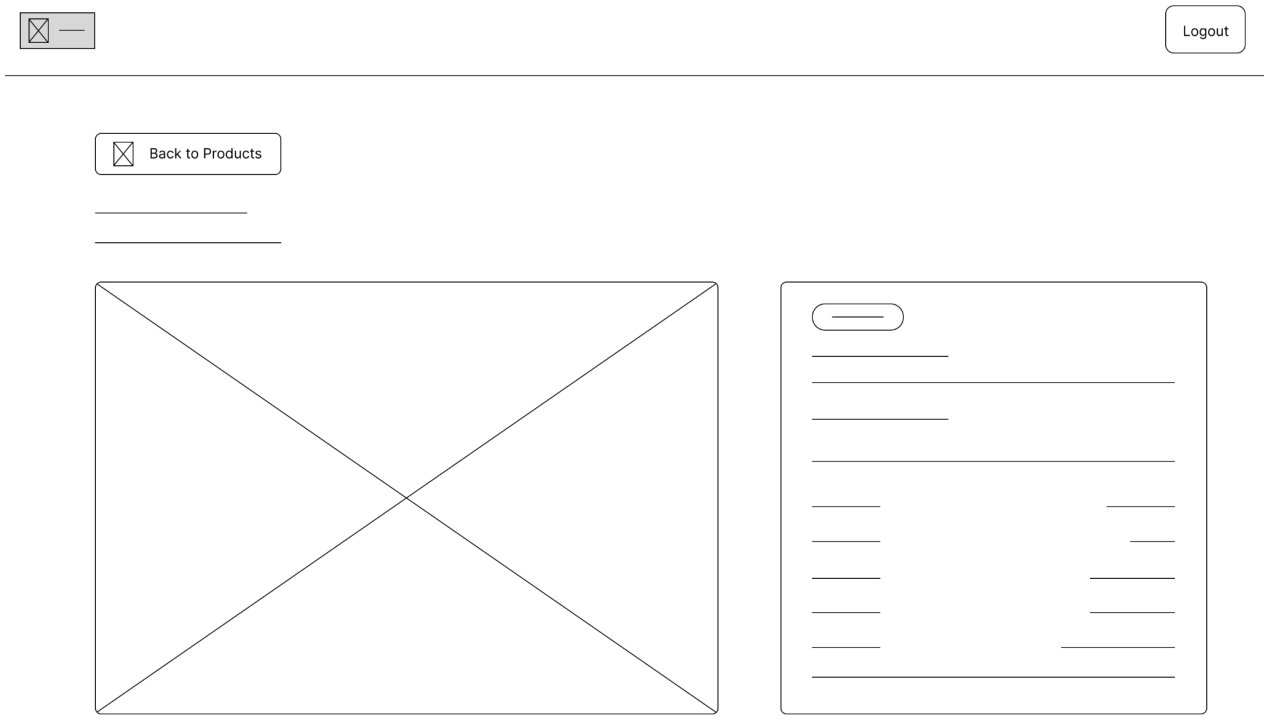


Admin Order Management Page

Admin User Management Page



Admin View Details Page



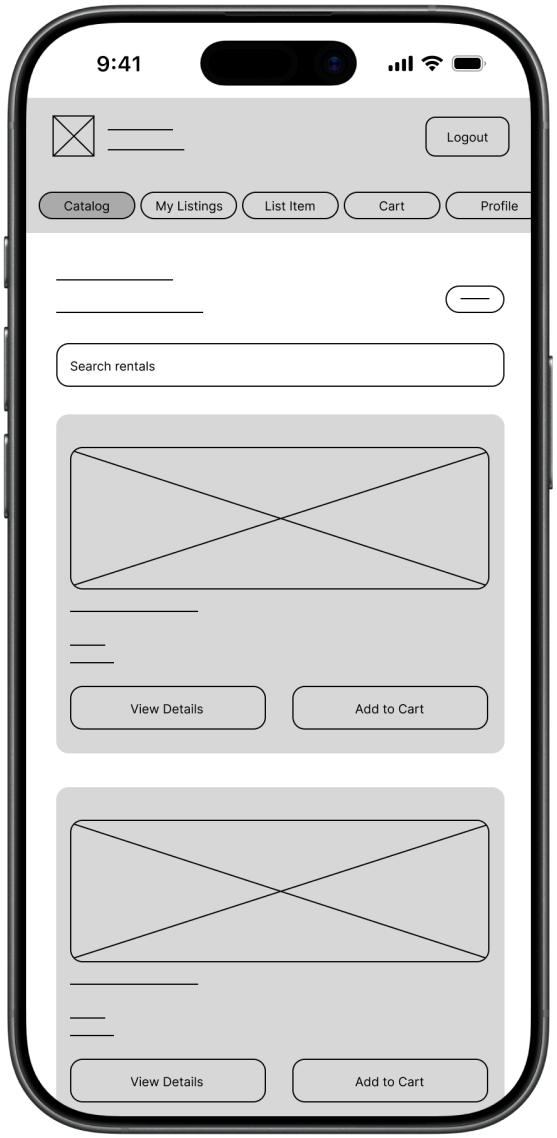
7.2 Mobile Application Wireframes

Register Page

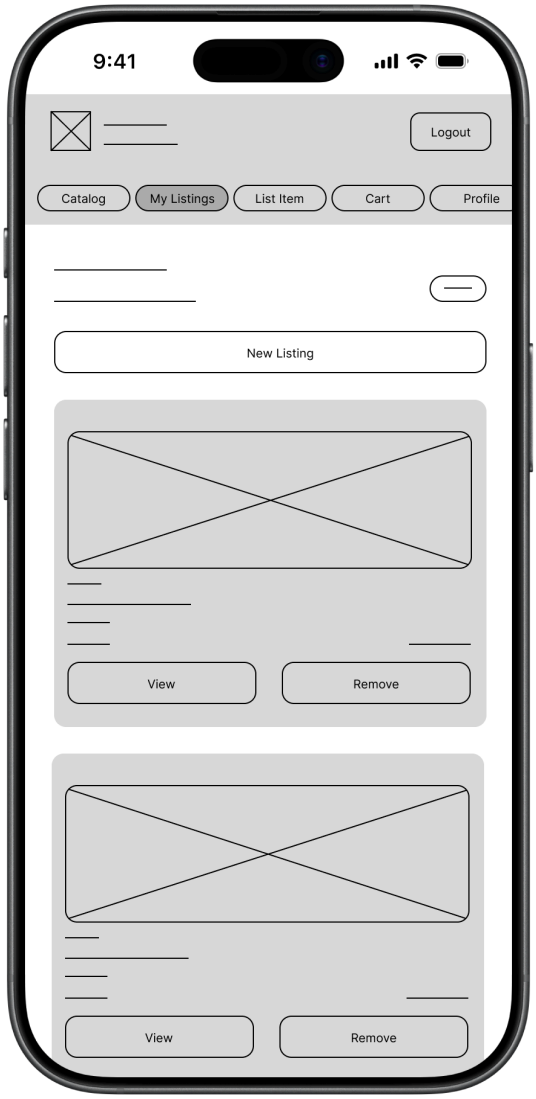
Login Page


9:41
Email
Enter your email address
Password
Create a Password
Login
or
Continue with Google
No account yet?
Create Account

User Home Page



User Listings Page

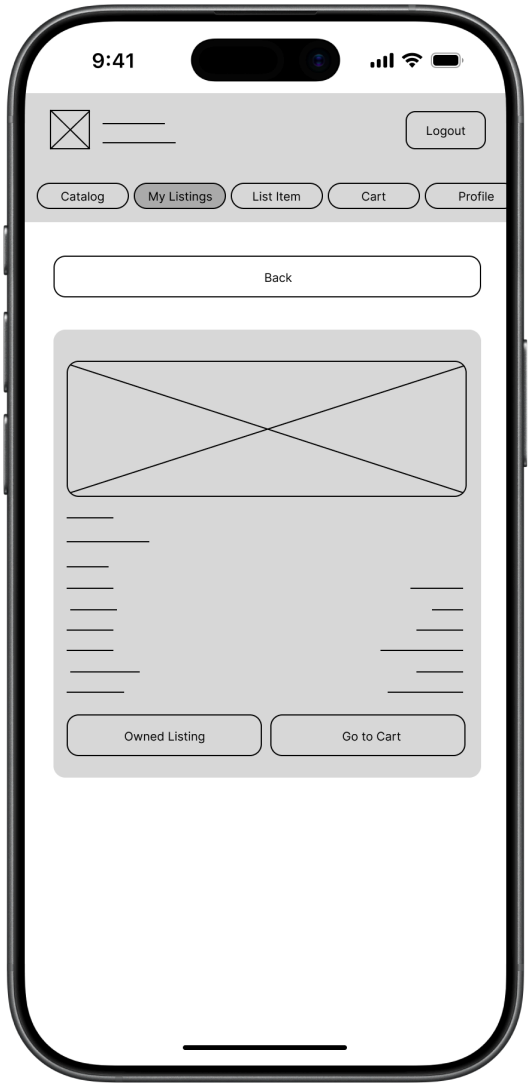


User List Items Page

User Cart Page

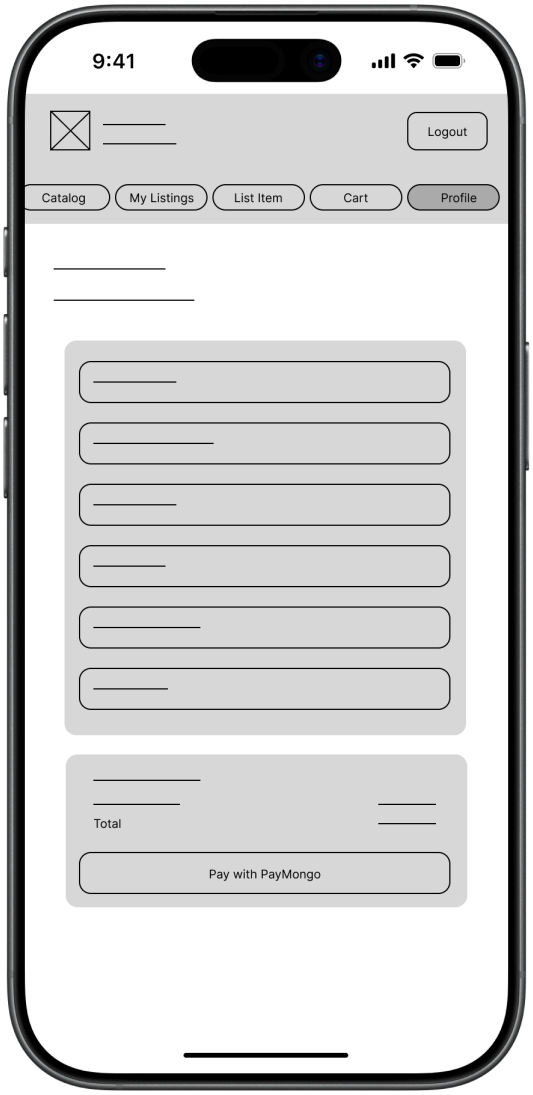
User Profile Page

View Details Page

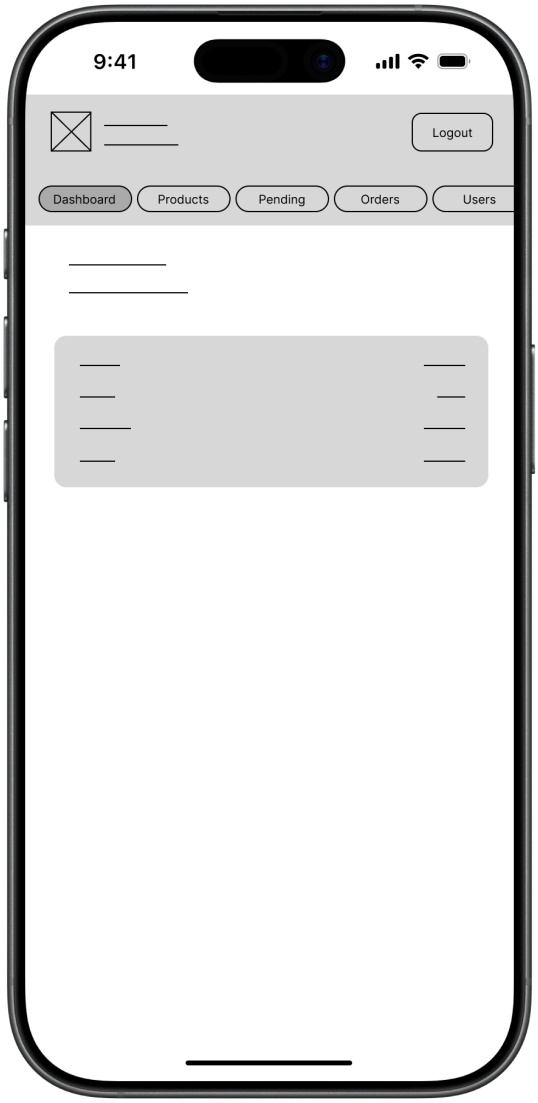


View Renter Profile Page

User Payment Page



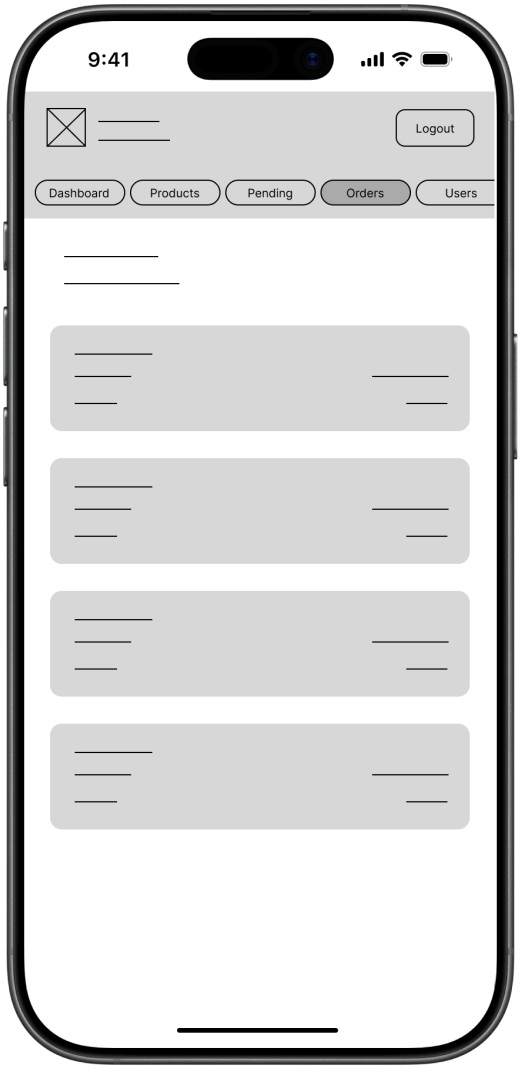
Admin Dashboard Page



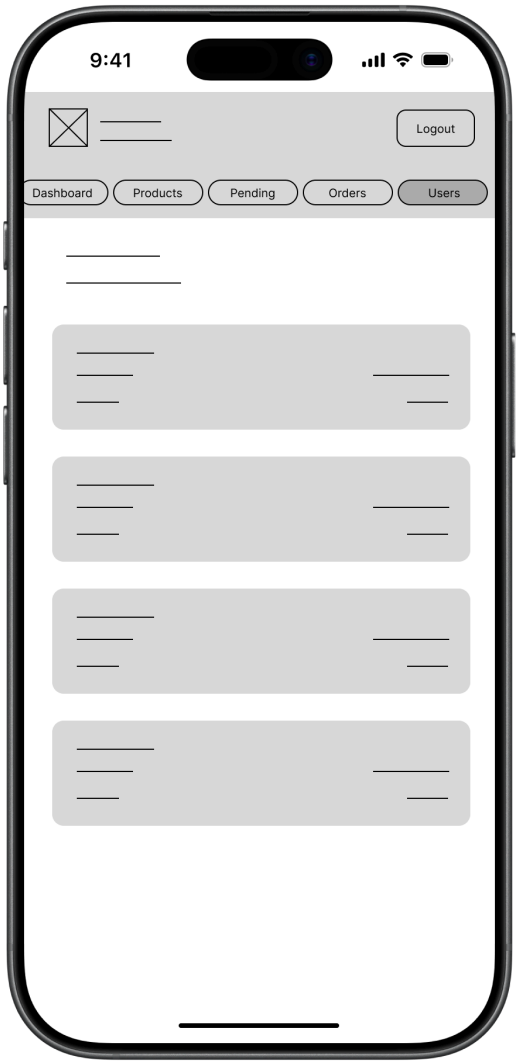
Admin Product Management Page

Admin Pending Approval Page

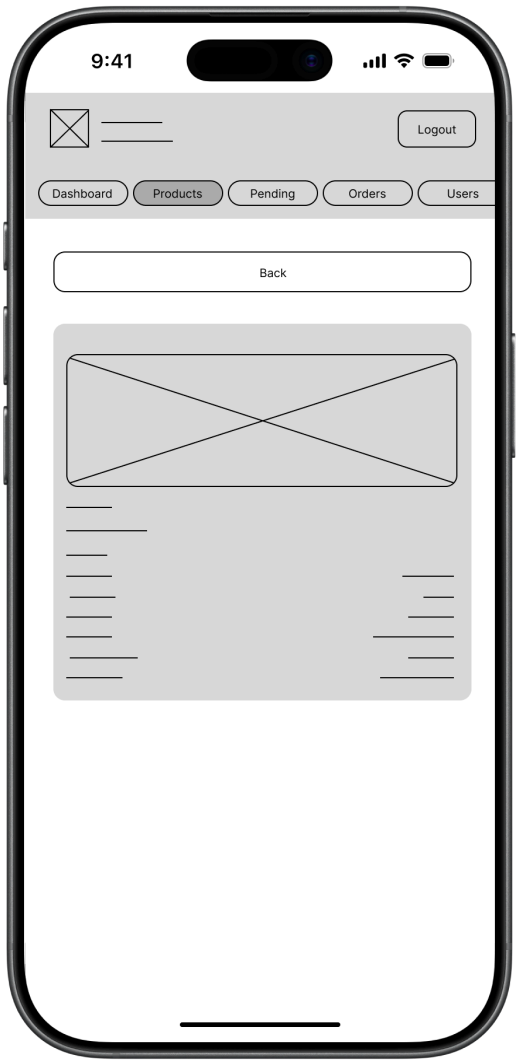
Admin Order Management Page



Admin User Management Page



Admin View Details Page



8.0 PLAN

8.1 Project Timeline Phase 1: Planning & Design (Week 1–2)

Phase 1: Planning & Design (Week 1–2)

Week 1: Requirements & Architecture

- Day 1–2: Project setup, initial documentation, and scope definition
- Day 3–4: Functional and non-functional requirements preparation
- Day 5–7: System architecture, component diagram, and technology stack identification

Week 2: Detailed Design

- Day 1–2: API contract and endpoint documentation
- Day 3–4: Database schema, ERD, and relationship finalization
- Day 5–6: UI/UX layout planning for customer, admin, and authentication pages
- Day 7: Review and revision of design documents

Phase 2: Backend Development (Week 3–4)

Week 3: Backend Foundation

- Day 1: Spring Boot project setup and dependency configuration
- Day 2: Supabase PostgreSQL connection and entity model creation
- Day 3: User registration, email/password login, Google authentication, JWT authentication, and BCrypt password hashing
- Day 4: Separate admin account structure and admin seeding
- Day 5: Product listing APIs and product approval status handling

Week 4: Core Backend Features

- Day 1: Cart API implementation with add, view, update rental days, and delete functions
- Day 2: Product deletion handling with related cart item cleanup
- Day 3: PayMongo checkout API integration
- Day 4: Error handling and backend testing
- Day 5: API documentation updates based on actual backend endpoints

Phase 3: Web Application (Week 5–6)

Week 5: Web Frontend Foundation

- Day 1: React, Vite, Tailwind CSS, and routing setup
- Day 2: Login and registration pages with Google sign-in support
- Day 3: Product catalog and product detail pages
- Day 4: Cart page with rental days and total calculation
- Day 5: Listing creation and my listings pages

Week 6: Complete Web Features

- Day 1: Checkout page and PayMongo checkout redirect
- Day 2: Order confirmation and backend order history display
- Day 3: User profile page with profile picture update
- Day 4: Admin dashboard, products, pending approval, orders, and users views
- Day 5: UI redesign, responsive layout improvements, and frontend-backend integration testing

Phase 4: Mobile Application (Week 7–8)

Week 7: Android Foundation

- Day 1: Android project setup using Kotlin
- Day 2: Retrofit client setup for backend API communication
- Day 3: Login and registration screens
- Day 4: Product browsing and product detail display
- Day 5: Cart API integration

Week 8: Complete Mobile App

- Day 1: Listing, cart, and checkout-related mobile screens
- Day 2: PayMongo checkout request integration
- Day 3: Admin/product management support on mobile
- Day 4: Emulator testing using local backend connection
- Day 5:Day 5: APK build preparation, mobile documentation, and mobile Google sign-in setup using Google Play Services Auth

Phase 5: Integration, Deployment & Finalization (Week 9–10)

Week 9: Integration Testing and Fixes

- Day 1: End-to-end testing across web, mobile, backend, and database
- Day 2: Cart, product deletion, and listing approval bug fixes
- Day 3: Database cleanup and ERD revision
- Day 4: API contract, requirements, and technology stack documentation updates
- Day 5: UI/UX review and final polish

Week 10: Deployment and Submission

- Day 1: Backend deployment on Render
- Day 2: Web frontend deployment on Vercel
- Day 3: Supabase PostgreSQL verification
- Day 4: Final system validation and screenshot collection
- Day 5: Final documentation, presentation preparation, and project submission

Risk Mitigation:

- Start with the simplest working version of each major feature
- Test backend, web, mobile, and database integration early
- Keep backups of working versions before database or schema changes
- Prioritize authentication, product catalog, cart, checkout, and admin approval features
- Use local testing before deploying to Render and Vercel
- Update documentation whenever implementation changes